

Software Requirements Specification

Agahee — Business Web Chat Application

(MERN Stack)

Business Web Chat Application Document Version: 1.1 (Upgraded)

Dev Team: Bilal / Saad / Ali

Client: Mr. Noor

Target Release Date: August 20th, 2026

1. Introduction	3
1.1 Purpose	3
1.2 Business & System Objectives	3
1.3 Scope	4
1.4 Definitions, Acronyms, and Abbreviations	4
2. Feasibility Study	4
2.1 Technical Feasibility	4
2.2 Operational Feasibility	5
2.3 Economic Feasibility	5
2.4 Legal & Compliance Feasibility	5
2.5 Schedule Feasibility	5
3. Overall Description	5
3.1 Product Perspective	5
3.2 Operating Environment	6
3.3 User Classes & Characteristics	6
3.4 Product Functions	6
3.5 Constraints	6
3.6 Assumptions and Dependencies	6
3.7 Work Breakdown Structure (WBS)	7
4. Specific Requirements	9
4.1 Functional Requirements	9
4.1.1 User Registration	9
4.1.2 Login / Logout & Authentication	9
4.1.3 Forgot / Reset Password	9
4.1.4 Search Users	9
4.1.5 One-to-One Chat & Live Messaging	9
4.1.6 Conversation List & Message History	9
4.1.7 Database Storage	10
4.1.8 Error Handling	10
4.1.9 Document / File Sharing	10
4.2 Non-Functional Requirements.	10
5. System Interfaces & Software Quality (SQ) Attributes	10
6. Use Cases	11
6.1 Use Case Summaries	11
6.2 Application Workflow Diagram	14
7. System Data Model	15
7.1 Entity Relationship Diagram (ERD):	15
7.2 Class Diagram	17
8. Tools, Technologies	18
8.1 Tools & Technology Justification	18
9. Medium-Priority Functional Requirements & SRS 2 Expansion	18

9.1 Release 2 Functional Requirements Specifications	19
9.1.1 Timestamps & Message Grouping	19
9.1.2 Unread Message Indicators	19
9.1.3 Read Receipts	19
9.1.4 Real-Time Typing Indicators	20
9.1.5 Online Status & Last Seen	20
9.1.6 Initials-Based Avatars	20
9.1.7 Message Input Validation & Alerts	20
9.1.8 Delete Message (Soft Delete)	21
End of SRS 1	21

SRS 1 (UPGRADED V1.1)

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) is to establish a clear, binding technical and operational foundation for Agahee, an enterprise web chat application. This document serves both technical team members (developers, software testers, project managers) and non-technical stakeholders (client Mr. Noor, business managers) to define exact system behavior, functional boundaries, operational constraints, and technology standards prior to implementation

1.2 Business & System Objectives

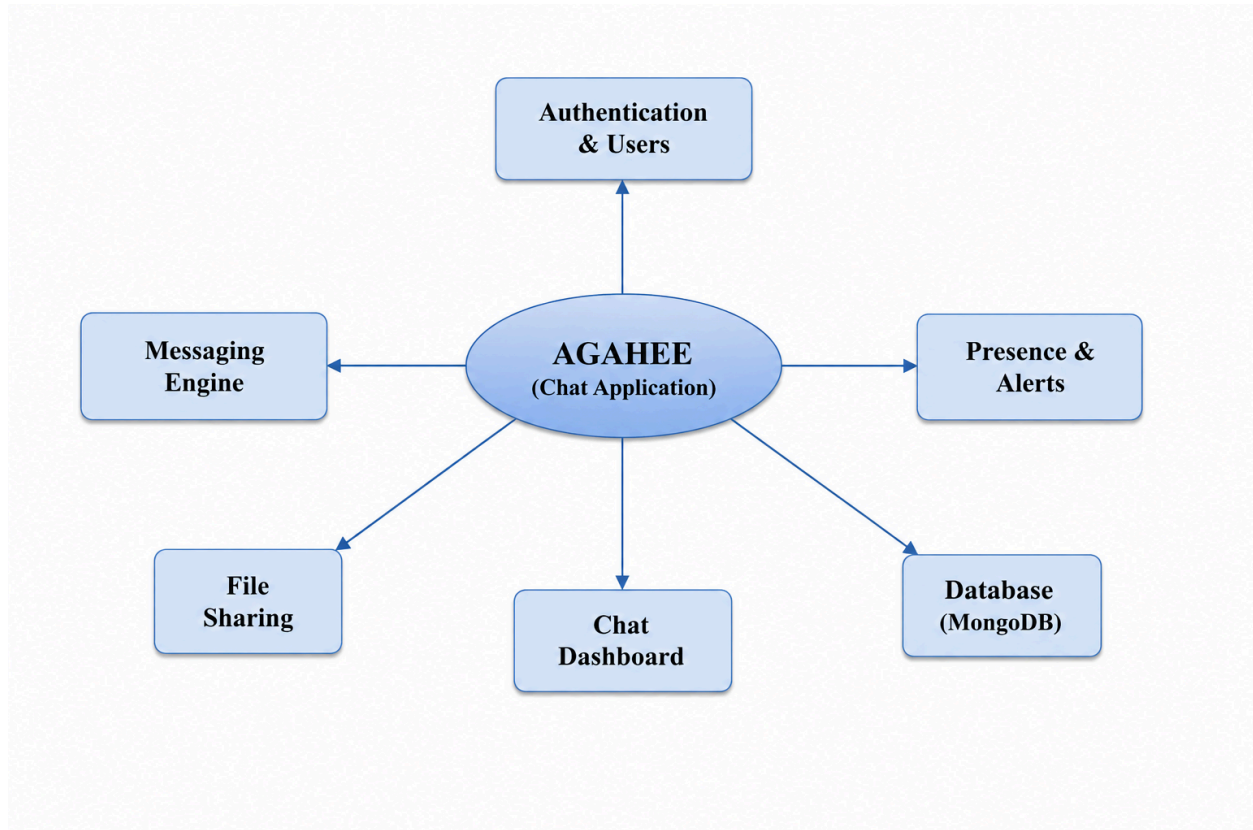
- **Productivity Enhancement:** Centralize team communication into a single internal platform to minimize email clutter and external app dependence.
- **Instantaneous Communication:** Deliver 1-to-1 messages with sub-second real-time latency (1s) under standard network conditions.
- **Secure Asset Sharing:** Provide safe, restricted document and file transmission within conversations.
- **High Availability:** Maintain a minimum 99% service availability standard during core corporate operating hours

1.3 Scope

Agahee is a web-based chat application built on the MERN stack (MongoDB, Express.js, React, Node.js) that allows registered users to communicate in real time. The system supports user registration and authentication, one-to-one private messaging, live message delivery via WebSockets, conversation history, and document/file sharing within conversations.

The system is intended for internal team communication within the organization, facilitating secure, real-time collaboration among employees.

Out-of-Scope: Group chat rooms, broadcast channels, voice and video calling, screen sharing, third-party app webhooks, and public multi-tenant access.



1.4 Definitions, Acronyms, and Abbreviations

MERN — MongoDB, Express.js, React, Node.js

JWT — JSON Web Token, used for stateless authentication

SRS — Software Requirements Specification

UI — User Interface

FR — Functional Requirement

NFR — Non-Functional Requirement

2. Feasibility Study

Before initiating software development, a five-dimensional feasibility assessment was conducted to confirm project viability.

2.1 Technical Feasibility

The chosen technology stack—**React, Node.js, Express.js, MongoDB, TypeScript, and Socket.io**—is highly mature and ideal for real-time web chat systems. Node.js utilizes an asynchronous, non-blocking I/O event loop that efficiently handles thousands of simultaneous WebSocket connections with low memory usage. React enables dynamic UI updates without full page refreshes. TypeScript provides strong type-checking to prevent runtime errors during development.

2.2 Operational Feasibility

Agahee is designed for immediate usability by internal organization employees. The user interface mimics modern industry-standard chat platforms (e.g., Slack, WhatsApp Web), requiring zero formal technical training for employees to send messages, search colleagues, or share files.

2.3 Economic Feasibility

The project utilizes 100% open-source core libraries and frameworks (MERN Stack, Socket.io), completely eliminating third-party software licensing costs. Deployment costs are minimized by using standard cloud hosting platforms (such as Render, Vercel, or AWS instances), keeping operational maintenance overhead low.

2.4 Legal & Compliance Feasibility

The application stores company communication internally and uses industry-standard encryption standards (JWT auth, salted Bcrypt password hashing, HTTPS/WSS transport encryption). It ensures complete compliance with workplace data privacy and corporate communication boundaries.

2.5 Schedule Feasibility

The project execution plan is structured into defined sprints covering MVP Core Messaging, Usability Enhancements, QA Testing, and Deployment. The milestone timeline ensures full delivery prior to the target deadline of **August 20th, 2026**.

3. Overall Description

3.1 Product Perspective

Agahee is a single-page, multi-tier web application using client-server architecture:

- **Presentation Tier:** Single Page Application (SPA) built with React and TypeScript.
- **Application/Logic Tier:** Node.js server with Express web APIs and a Socket.io event-handling gateway.
- **Database Tier:** MongoDB Atlas document repository.

3.2 Operating Environment

- **Client Side:** Modern, standard web browsers (Google Chrome v100+, Mozilla Firefox v100+, Apple Safari v15+, Microsoft Edge v100+) on Desktop.
- **Server Infrastructure:** Node.js runtime environment (v18.x LTS or higher) hosted on Linux/Cloud App Servers.
- **Database Infrastructure:** MongoDB cluster (v6.0 or higher) with indexing enabled on User and Message collections.

3.3 User Classes & Characteristics

- **Standard Registered Employee (Primary Actor):** Non-technical end users who utilize the chat app to collaborate, search colleagues, exchange messages, and send documents. Requires minimal tech literacy.

3.4 Product Functions

At a high level, Agahee allows a user to:

- Register and securely log into an account
- Search for and start conversations with other registered users
- Send and receive text messages in real time
- View a list of ongoing conversations and past message history
- Share documents/files within a conversation
- Reset a forgotten password

3.5 Constraints

- The system must be built using the **MERN** stack.
- Authentication must use **JWT-based**, token-based sign-in.
- All user and message data must be persisted in **MongoDB**.
- The system must utilize **Socket.io** to enable real-time, bidirectional messaging between users.
- The codebase must be developed using **TypeScript** instead of standard JavaScript for enhanced type safety and maintainability.
- The project must be completed and delivered by the final deadline of **August 20th, 2026**.

3.6 Assumptions and Dependencies

- Users have access to a modern web browser and a stable internet connection.
- The system depends on third-party libraries/services such as Socket.io for real-time messaging.

3.7 Work Breakdown Structure (WBS)

Work Breakdown Structure (WBS): A hierarchical decomposition of the entire project scope into manageable deliverables. It outlines project phases including Project Management & Setup, Core MVP Messaging & Auth (High Priority), Usability Enhancements (Medium Priority), Advanced Features (Low Priority), and QA, Deployment & Documentation.

WBS Code	Level 1	Level 2	Level 3	Estimated Hours
1.0	1.0 Project Management			
1.1		1.1 Planning & Setup		
1.1.1			Requirements & Scope Finalization	1

1.1.2			Tech Stack Setup (MERN + Socket.io)	1
1.1.3			System & DB Architecture Design	4
1.1.4			UI/UX Wireframing	3
2.0	2.0 Authentication & Users			
2.1		2.1 Authentication		
2.1.1			User Registration	1
2.1.2			User Authentication	1
2.1.3			User Authorization	1
2.2		2.2 User Management		
2.2.1			Block/Unblock	2
2.2.2			Search User	1
2.3		2.3 Data Handling		
2.3.1			User Data Storage	4
2.3.2			User Data Processing	3
2.3.3			User Data Security	2
2.3.4			User Data Verification	2

3.0	3.0 Usability Enhancements			
3.1		3.1 UI Polish		
3.1.1			UI Polish	4
4.0	4.0 Advanced Features			
4.1		4.1 Messaging Engine		
4.1.1			Message Replying	2
4.1.2			Message Forward	2
4.1.3			Message Delete	2
4.1.4			Message React	2
4.1.5			Message Pin	2
4.1.6			Document Sharing	
4.2.0		4.2 Platform Foundation		
4.2.1			Socket.io Configuration	2
4.2.2			Express.js server	2
4.2.3			REST API	2
4.2.4			MongoDB integration	1

4.2.5			Environment Configuration	2
4.2.6			Login and error handling	2
5.0.0	5.0 QA & Docs			
5.1.0		5.1 Quality		
5.1.1			SQA Testing	4
5.2.0		5.2 Documentation		
5.2.1			Documentation	4
6.0.0	6.0 Presence & Alerts			
6.1.0		6.1 Presence & Alerts		
6.1.1			Alerts	1
6.1.2			Status	2
7.0.0	7.0 AI Chatbot			
7.1.0		7.1 AI Chatbot		
7.1.1			Summarization	2
7.1.2			Message Suggestions	2

4. Specific Requirements

This section lists the High Priority features from the client's feature list, expressed as testable functional requirements. Medium and Low priority features will be incorporated in SRS 2.

4.1 Functional Requirements

4.1.1 User Registration

ID	Requirement
FR-1	The system shall allow a new user to register an account by providing a name, email, and password.
FR-2	The system shall validate that the email provided during registration is properly formatted.

4.1.2 Login / Logout & Authentication

ID	Requirement
FR-3	The system shall allow a registered user to log in using their email and password.
FR-4	The system shall issue a JWT upon successful login.
FR-5	The system shall hash and salt all user passwords before storing them in the database.

4.1.3 Forgot / Reset Password

ID	Requirement
FR-6	The system shall allow a user to request a password reset link via their registered email.
FR-7	The system shall allow a user to set a new password.

4.1.4 Search Users

ID	Requirement
FR-8	The system shall allow a logged-in user to search for other registered users.

4.1.5 One-to-One Chat & Live Messaging

ID	Requirement
FR-9	The system shall allow a user to start a private, one-to-one conversation with another registered user.
FR-10	The system shall deliver messages between users in real time using WebSockets (Socket.io).
FR-11	The system shall allow a user to send and receive text messages within a conversation.

4.1.6 Conversation List & Message History

ID	Requirement
FR-12	The system shall display a list of all conversations the logged-in user is part of.
FR-13	The system shall allow a user to view the full message history of a past conversation.

4.1.7 Database Storage

ID	Requirement
FR-14	The system shall persist all user account data and messages in a MongoDB database.

4.1.8 Error Handling

ID	Requirement
FR-15	The system shall display consistent, user-friendly error messages for failed actions across the application.

4.1.9 Document / File Sharing

ID	Requirement
FR-16	The system shall allow a user to share a document or file within a conversation.

4.2 Non-Functional Requirements

- **NFR-1 (Performance):** The system shall deliver real-time messages via WebSockets (Socket.io) to online recipients within 1 second under normal network conditions.
- **NFR-2 (Security):** User passwords shall be stored using hashing and salting, and all API communication and WebSocket connections shall be secured using HTTPS/WSS.
- **NFR-3 (Usability):** The application interface shall be intuitive and responsive, enabling internal business team members to send messages and share files without requiring prior training.
- **NFR-4 (Reliability & Availability):** The application shall maintain 99% uptime during core business hours to ensure uninterrupted internal team communication.
- **NFR-5 (Maintainability):** The codebase shall be strictly typed using TypeScript across both front-end and back-end components to improve code quality and facilitate future development.

5. System Interfaces & Software Quality (SQ) Attributes

- **User Interface (UI):** Single Page Application featuring Login/Register view, Left Sidebar (Search, Contact List), Main Panel (Active Chat, Socket Connection Status indicator).
- **Hardware Interfaces:** Minimum client RAM 2GB; Minimum server hardware 1 vCPU, 2GB RAM.
- **Software Interfaces:** Integration with MongoDB database drivers, Express API router, Node.js HTTP server, and Socket.io engine

6. Use Cases

- **Primary Actor: Registered Team Member** (an authenticated internal employee using the web app)

6.1 Use Case Summaries

- Register Account
- Log In / Log Out
- Reset Password
- Search for a User
- Start a Conversation
- Send / Receive a Message
- View Conversation History
- Share a File in a Conversation

Attached images are the use case diagrams for account access and messaging/conversation respectively.

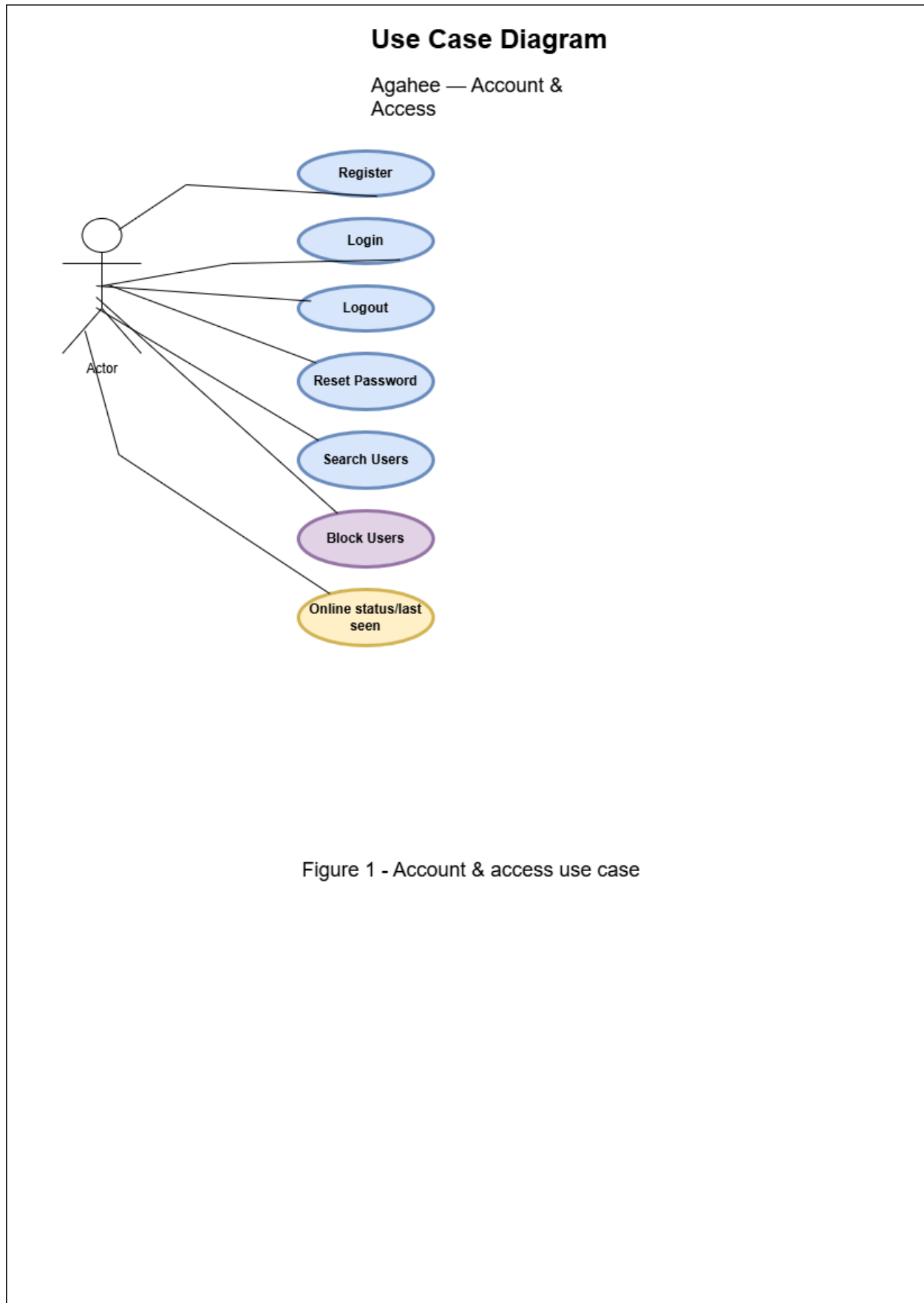
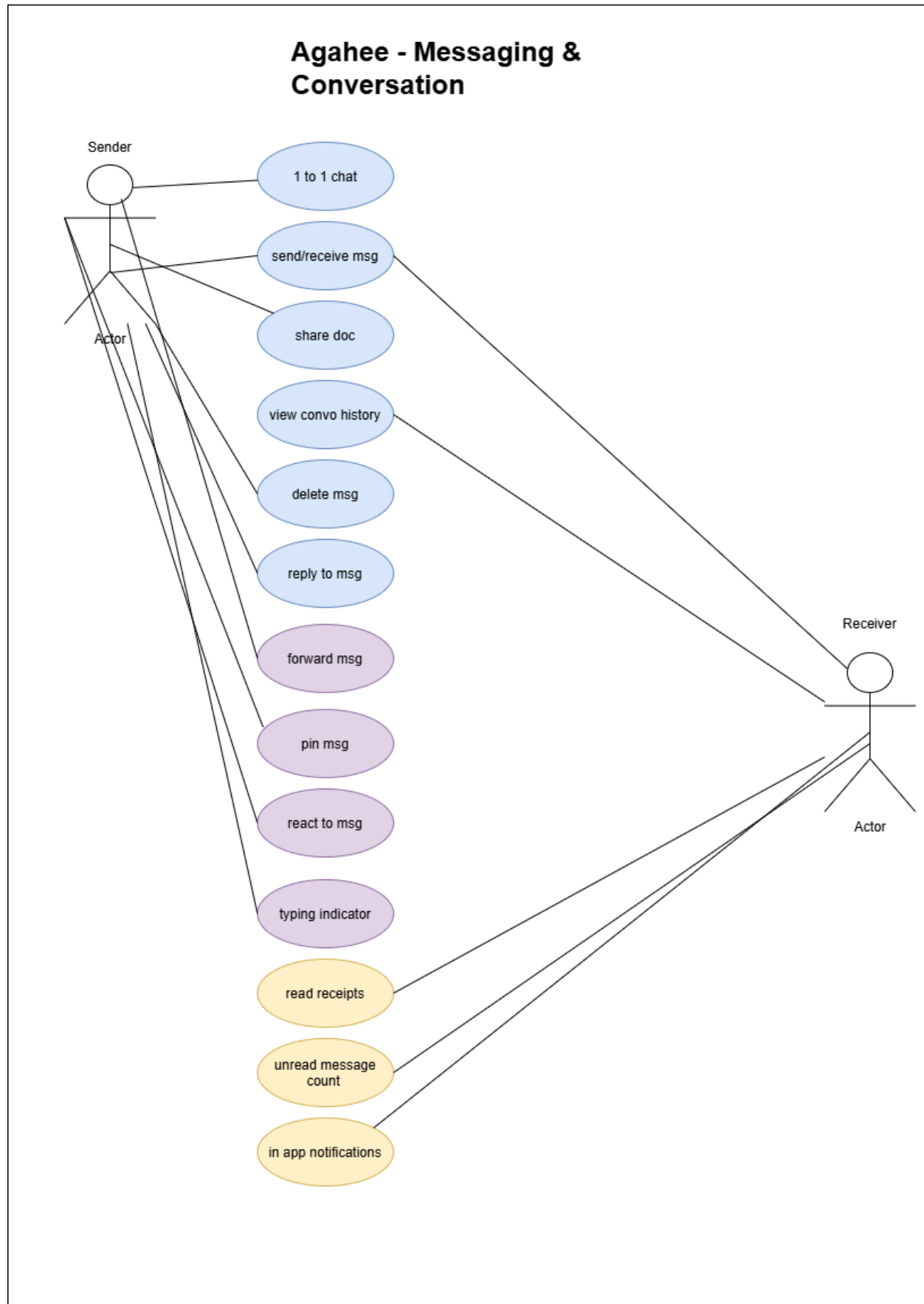
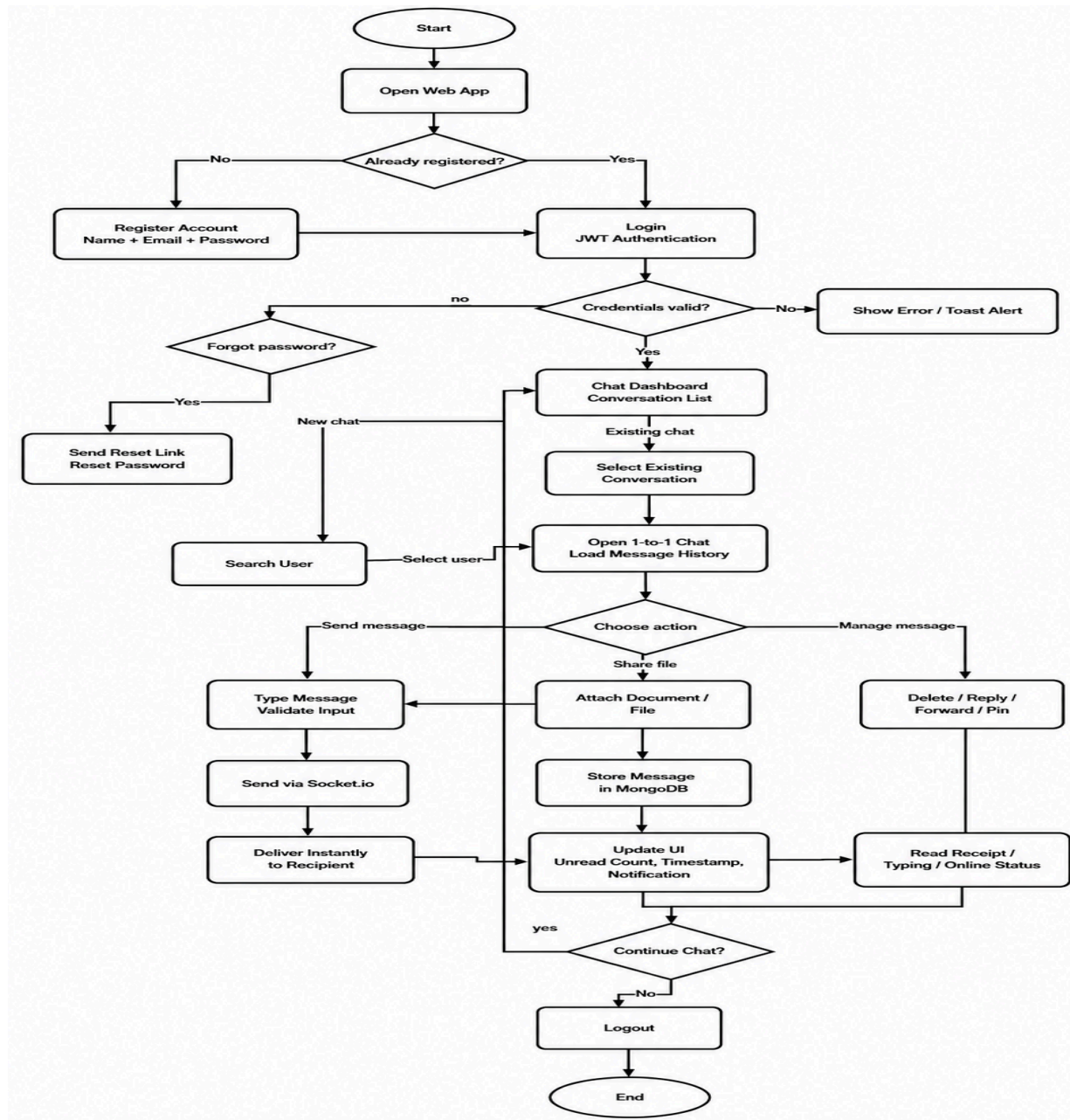


Figure 1 - Account & access use case



6.2 Application Workflow Diagram

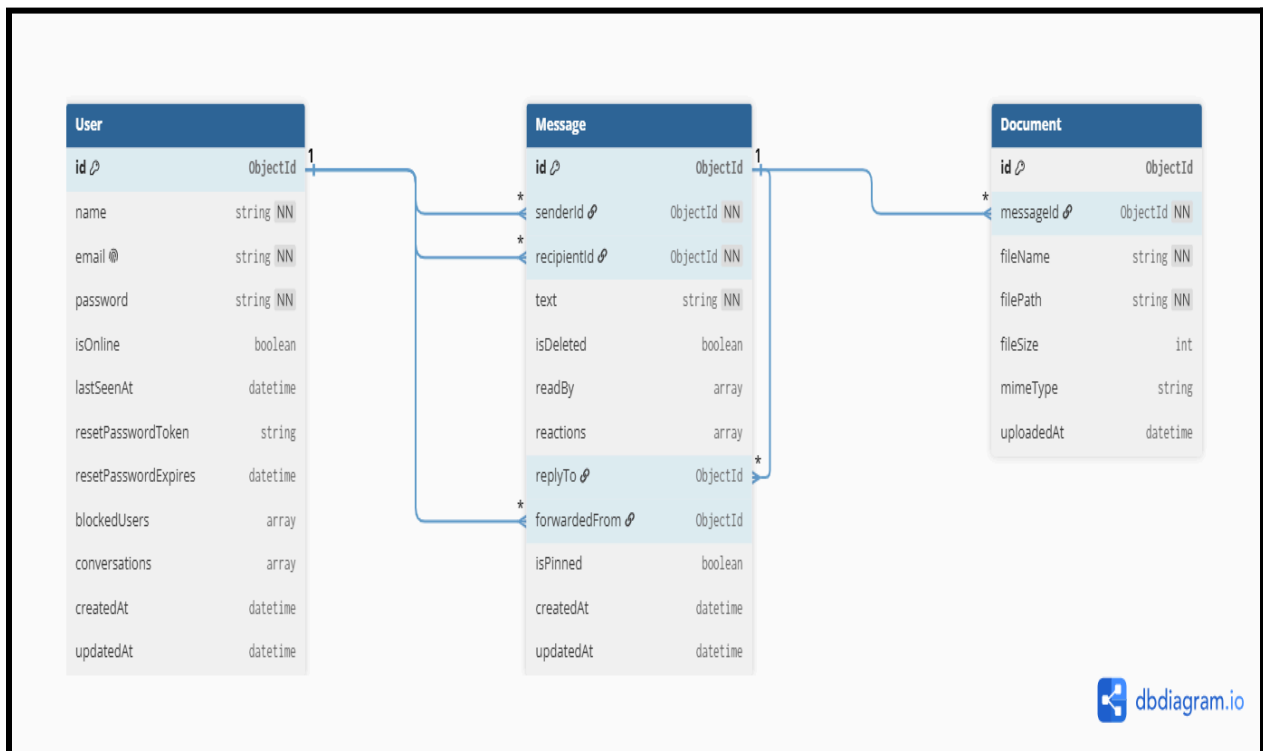
System Workflow Diagram: A step-by-step visual representation of the user journey within the Agahee application. It maps out authentication flows (Registration, Login, Password Reset), navigating to the Chat Dashboard, initiating search or existing chats, sending real-time messages via Socket.io, attaching documents, updating MongoDB storage, and ending session actions.



7. System Data Model

7.1 Entity Relationship Diagram (ERD):

A visual representation of the system's data model, showing the core entities (User, Conversation, Message, Document), their attributes, and the relationships between them. It documents One-to-Many relationships (e.g. a Conversation has many Messages) and Many-to-Many relationships (e.g. Users and Conversations, resolved via a junction table) that structure how data is stored and connected throughout the application.



Agahee keeps track of information using three main categories: **Users**, **Messages**, and **Documents**. Think of these as three separate filing cabinets, each holding one kind of record, with simple connections between them showing how they relate.

Users — This cabinet holds one record per person who has an account: their name, email, password, and whether they're currently online. It also keeps a short note on who each user has chosen to block, and a quick reference list of who they've been chatting with recently, this second part is just a shortcut for showing the conversation list quickly, not the actual message content.

Messages — Every message sent through the app gets its own record here: who sent it, who it was sent to, the text itself, and when it was sent. A message also keeps a short list of who has read it (for read receipts) and any quick reactions (like a thumbs up) it received. If a message was a reply to an earlier one,

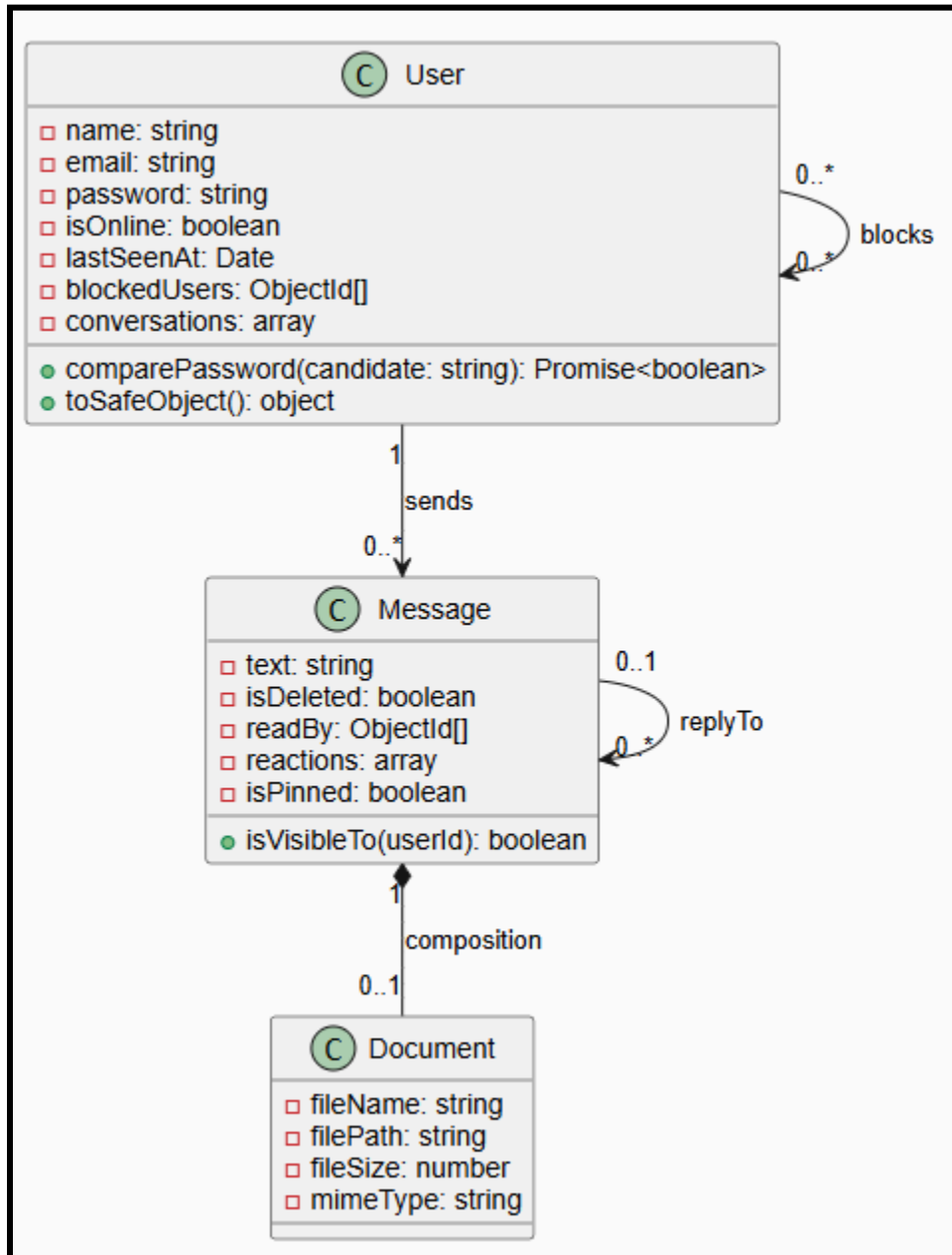
or was forwarded from somewhere else, that's noted too. Deleted messages aren't erased, they are simply marked as deleted so the system remembers a message existed there, without showing its content anymore.

Documents — When someone shares a file in a conversation, a record is created here linking that file back to the specific message it was shared in.

How they connect:

- Every message is linked to exactly two people, whoever sent it, and whoever it was sent to. There's no separate "conversation" record at all; a conversation between two people is simply the full set of messages exchanged between them, gathered together automatically.
- A Document is always linked to the one Message it was shared with.
- A Message can optionally point back to another Message, when it's a reply

7.2 Class Diagram



While the data model shows *what information is stored*, this diagram shows *what each part of the system is responsible for doing*, the building blocks the application is made of, and what each one is allowed to do with its own information.

There are three building blocks: **User**, **Message**, and **Document**, matching the same three categories from the data model, but here we also see their **behaviors**, shown at the bottom of each box.

User — Represents a person using the app. Beyond just holding someone's name, email, and password, a User is capable of two specific things on its own: it can check whether a given password is correct (used

when someone logs in), and it can prepare a "safe" version of itself to send back to the app's screen, one that leaves out the password, so sensitive information never accidentally gets displayed.

Message — Represents a single message sent between two people. It's capable of one thing on its own: determining whether it should currently be visible to a particular person, taking into account whether the message has been deleted. Everything else about how messages get created, sent, or organized is handled elsewhere in the system, not by the message itself.

Document — Represents a shared file. Unlike the other two, it doesn't do anything on its own; it simply holds file details.

How they relate to each other:

- A User **sends** many Messages, shown by the arrow between them one User can send any number of messages over time.
- A Message can **reply** to another Message shown by the loop pointing back to itself since a reply is just a message referencing an earlier one.
- A User can **block** other Users shown by the loop on the User box a person can block, or be blocked by, any number of other people.
- A Document belongs to exactly one Message through a relationship called **composition** shown by the solid connecting line meaning a Document only exists because a Message shared it; it isn't a standalone piece of the system on its own.

8. Tools, Technologies

8.1 Tools & Technology Justification

- **React + TypeScript:** Enables fast, component-driven UI development with compile-time type validation, eliminating runtime UI crashes.
- **Node.js + Express.js:** Handles incoming API endpoints and concurrent socket connections seamlessly.
- **Socket.io:** Automates real-time fallback mechanisms (e.g., long polling) if WebSockets are blocked by strict enterprise firewalls.
- **MongoDB:** Document-based storage matches JavaScript objects natively, simplifying dynamic chat schema modifications.

9. Medium-Priority Functional Requirements & SRS 2 Expansion

This section formally incorporates the medium-priority functional features into the system requirements baseline. It outlines both the immediate implementation specifications and the architectural expansion plan to be fully realized in **SRS 2**.

9.1 Release 2 Functional Requirements Specifications

9.1.1 Timestamps & Message Grouping

ID	Requirement
FR-17	The system shall display the exact send time for all chat messages.
FR-18	The system shall group conversation messages under dynamic date.

9.1.2 Unread Message Indicators

ID	Requirement
FR-19	The system shall display an unread message badge count on conversation list threads containing unread messages.
FR-20	The system shall clear the unread counter when the user opens the corresponding conversation.

9.1.3 Read Receipts

ID	Requirement
----	-------------

FR-21	The system shall update message status indicators to show when a message has been viewed by the recipient.
FR-22	The system shall emit real-time socket events to update read receipts without requiring a page refresh.

9.1.4 Real-Time Typing Indicators

ID	Requirement
FR-23	The system shall display a typing indicator (e.g., "User is typing...") when a participant is actively typing a message.
FR-24	The system shall remove the typing indicator when the user stops typing or sends the message.

9.1.5 Online Status & Last Seen

ID	Requirement
FR-25	The system shall display a visual presence indicator showing whether a user is currently online or offline.
FR-26	The system shall record and display the last active timestamp when a user disconnects.

9.1.6 Initials-Based Avatars

ID	Requirement
----	-------------

FR-27	The system shall automatically generate dynamic profile avatars displaying user initials when no custom avatar image is set.
-------	--

9.1.7 Message Input Validation & Alerts

ID	Requirement
FR-28	The system shall validate message payloads to prevent sending empty text or whitespace-only messages.
FR-29	The system shall display non-intrusive toast alerts for system errors, failed actions, or background messages.

9.1.8 Delete Message (Soft Delete)

ID	Requirement
FR-30	The system shall allow a user to delete any message from their active chat window.

End of SRS 1

SRS 2 will incorporate Medium Priority features (timestamps, unread counts, read receipts, typing indicator, online status/last seen, initials-based avatars, message validation, toast alerts, delete message, in-app notifications) as additional functional requirements, along with expanded diagrams and refined non-functional requirements, once feedback on this draft is received.

